SIMATS ENGINEERING
ASSESSMENT TOOL 3

COURSE NAME: CRYPTOGRAPHY AND NETWORK SECURITY
COURSE CODE: CSA51

COURSE OUTCOME COVERED

CO2: Analyze Public Key crypto system strategies using RSA and key Exchange for Encryption algorithm to strengthen information security. (BL4,BL5)

Debugging & Optimisation Task : 10%

QUESTIONS: **Total Marks: 50**

Annexure A

Code of Conduct:

I **D. Naveen / 192372177** *(Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.*

Signature of the student:

Annexure A

Debugging & Optimisation Task

1. Debugging Hash Function Implementation (SHA Family)

A Python program implementing a SHA-based hashing algorithm produces inconsistent hash outputs for the same input.

Identify errors in message padding and block processing steps.
Fix issues related to bitwise operations and endianness handling.
Optimize the implementation to improve processing speed for large files (≥50 MB).
Evaluate how secure hash functions improve integrity compared to basic checksum methods.

2. Optimization of Secure Socket Layer (SSL/TLS) Simulation

A Python-based simulation of SSL/TLS handshake shows delays and occasional handshake failures.

Debug issues in certificate validation and key exchange steps.
Optimize the handshake process to reduce latency.
Refactor the code to reuse session keys efficiently (session resumption).
Analyze the performance vs security trade-offs in TLS optimization.

3. Debugging and Optimizing Digital Signature Algorithm (DSA)

A Python implementation of DSA generates invalid signatures during verification.

Identify issues in parameter selection and random number generation.
Fix errors in signature generation and verification steps.
Optimize modular arithmetic operations for faster execution.
Evaluate the impact of randomness quality on signature security.

4. Debugging Key Management System

A Python-based key management system fails to securely store and retrieve cryptographic keys.

Identify flaws in key storage (plaintext storage, improper encryption).
Fix issues related to key lifecycle management (generation, distribution, rotation).
Optimize secure storage using encryption and access control mechanisms.
Analyze how poor key management can compromise an entire cryptographic system.

5. Optimization of Secure File Transfer Protocol (SFTP-like System)

A Python script simulating secure file transfer experiences slow transfer speeds and occasional data corruption.

Debug issues in encryption/decryption synchronization.
Fix errors in packet handling and integrity verification.
Optimize throughput for large file transfers (≥100 MB).
Compare the optimized system with insecure file transfer methods and justify the improvements.

RUBRICS FOR CALCULATION

Criteria	Weightage	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)
Problem Identification	20%	Accurately identifies all errors and root causes with clear understanding	Identifies most errors with minor gaps	Identifies some errors but misses key issues	Unable to identify errors correctly
Debugging	25%	Applies systematic debugging techniques effectively to resolve issues	Uses appropriate debugging methods with minor errors	Limited debugging approach; requires guidance	Unable to debug or resolve issues
Optimization	20%	Optimizes code by significantly improving time complexity using efficient algorithms	Improves time complexity with minor gaps in efficiency	Limited improvement in time complexity	No improvement; inefficient approach retained
Analysis	20%	Demonstrates strong logical reasoning and clear justification of solutions	Shows reasonable analysis with some justification	Limited analysis; weak reasoning	No analytical reasoning demonstrated
Code Quality	15%	Produces clean, well-structured code with proper comments and	Code is mostly clear with minor documentation gaps	Code lacks structure and proper documentation	Poorly written code with no documentation

		documentation			

1. Debugging Hash Function Implementation (SHA Family)

*The faulty source file is not included in the question paper, so the defects below are the standard implementation errors that produce exactly the symptom described (different digests for the same input). Each one is fixed in the corrected program (file **q1_sha256.py**), which was tested against Python's* `hashlib`.

(a) Errors identified in padding and block processing

#	Error (root cause)	Fix
1	Padding: **0x80** delimiter byte missing / zero bytes used. Messages that differ only by trailing zeros collide, and the length is not aligned.	Append `0x80`, then zeros until length ≡ 56 (mod 64).
2	Length field written in **bytes** and/or **little-endian** (`'<Q'`).	Append the **bit** length (8×len) as 64-bit **big-endian** (`'>Q'`).
3	Words read with `'<16I'` or native order (`'@I'`). SHA-2 is big-endian, so the result changes with the machine's byte order.	`struct.unpack('>16I', block)` and `pack('>8I', ...)` for the output.
4	Bitwise: rotation coded as a shift (`x >> n`) instead of ROTR, which loses the bits that wrap around.	`(x >> n | x << (32-n)) & 0xFFFFFFFF`
5	Python integers are unbounded: additions and left shifts are not truncated to 32 bits.	Mask with `& 0xFFFFFFFF` after every add / shift.
6	Block processing: bytes left over between `update()` calls are dropped, so the digest depends on how the file is chunked.	Keep a remainder buffer; compress only whole 64-byte blocks.
7	`digest()` mutates the internal state, so calling it twice gives different output.	Pad a copy of the state; restore it afterwards.

(b) Corrected and optimised implementation

```
"""Q1: SHA-256 - fixed (padding, endianness, bit ops) and optimised."""
import struct, hashlib, time, os

K = (0x428a2f98,0x71374491,0xb5c0fbcf,0xe9b5dba5,0x3956c25b,0x59f111f1,0x923f82a4,0xab1c5ed5,
0xd807aa98,0x12835b01,0x243185be,0x550c7dc3,0x72be5d74,0x80deb1fe,0x9bdc06a7,0xc19bf174,
0xe49b69c1,0xefbe4786,0x0fc19dc6,0x240ca1cc,0x2de92c6f,0x4a7484aa,0x5cb0a9dc,0x76f988da,
0x983e5152,0xa831c66d,0xb00327c8,0xbf597fc7,0xc6e00bf3,0xd5a79147,0x06ca6351,0x14292967,
0x27b70a85,0x2e1b2138,0x4d2c6dfc,0x53380d13,0x650a7354,0x766a0abb,0x81c2c92e,0x92722c85,
0xa2bfe8a1,0xa81a664b,0xc24b8b70,0xc76c51a3,0xd192e819,0xd6990624,0xf40e3585,0x106aa070,
0x19a4c116,0x1e376c08,0x2748774c,0x34b0bcb5,0x391c0cb3,0x4ed8aa4a,0x5b9cca4f,0x682e6ff3,
0x748f82ee,0x78a5636f,0x84c87814,0x8cc70208,0x90befffa,0xa4506ceb,0xbef9a3f7,0xc67178f2)
H0 = (0x6a09e667,0xbb67ae85,0x3c6ef372,0xa54ff53a,0x510e527f,0x9b05688c,0x1f83d9ab,0x5be0cd19)
M = 0xFFFFFFFF                                  # FIX 1: keep every word 32-bit

class SHA256:
    def __init__(self):
        self.h = list(H0); self.buf = b""; self.n = 0
    def _compress(self, blk):
        # FIX 2: message words are BIG-endian (">16I"), not little-endian
        w = list(struct.unpack(">16I", blk)) + [0] * 48
        for i in range(16, 64):                 # OPT: rotations inlined (no function call)
            x = w[i-15]; y = w[i-2]
            s0 = ((x >> 7 | x << 25) ^ (x >> 18 | x << 14) ^ (x >> 3)) & M
            s1 = ((y >> 17 | y << 15) ^ (y >> 19 | y << 13) ^ (y >> 10)) & M
            w[i] = (w[i-16] + s0 + w[i-7] + s1) & M
        a, b, c, d, e, f, g, h = self.h
        for i in range(64):
            S1 = ((e >> 6 | e << 26) ^ (e >> 11 | e << 21) ^ (e >> 25 | e << 7)) & M   # FIX 3: rotr, not shift
            ch = (e & f) ^ (~e & g)
            t1 = (h + S1 + ch + K[i] + w[i]) & M
            S0 = ((a >> 2 | a << 30) ^ (a >> 13 | a << 19) ^ (a >> 22 | a << 10)) & M
            mj = (a & b) ^ (a & c) ^ (b & c)
            t2 = (S0 + mj) & M
            h, g, f, e, d, c, b, a = g, f, e, (d + t1) & M, c, b, a, (t1 + t2) & M
        self.h = [(x + y) & M for x, y in zip(self.h, (a, b, c, d, e, f, g, h))]
    def update(self, data):                     # OPT: streaming, constant memory
        self.n += len(data)
        data = self.buf + data
        end = len(data) - len(data) % 64
        mv = memoryview(data)
        for i in range(0, end, 64):
            self._compress(mv[i:i+64])
        self.buf = bytes(data[end:])
    def digest(self):
        # FIX 4: pad = 0x80, zeros, then 64-bit BIG-endian *bit* length
        tail = self.buf + b"\x80"
        tail += b"\x00" * ((56 - len(tail)) % 64) + struct.pack(">Q", self.n * 8)
        h0 = list(self.h)
        for i in range(0, len(tail), 64):
            self._compress(tail[i:i+64])
        out = struct.pack(">8I", *self.h); self.h = h0
        return out
    def hexdigest(self): return self.digest().hex()

def sha256_file(path, chunk=1 << 20, use_native=True):
    """Large-file hashing: 1 MiB chunks; OpenSSL (C) engine if allowed."""
    h = hashlib.sha256() if use_native else SHA256()
    with open(path, "rb", buffering=0) as f:
        while block := f.read(chunk):
```

```
            h.update(block)
    return h.hexdigest()

if __name__ == "__main__":
    for m in (b"", b"abc", b"a" * 55, b"a" * 56, b"a" * 64, os.urandom(1000)):
        s = SHA256(); s.update(m)
        assert s.hexdigest() == hashlib.sha256(m).hexdigest(), m[:10]
    s = SHA256(); [s.update(b"abc"[i:i+1]) for i in range(3)]       # chunk-independent
    assert s.hexdigest() == hashlib.sha256(b"abc").hexdigest()
    print("all test vectors OK")
    data = os.urandom(50 * 1024 * 1024)
    open("/tmp/big.bin", "wb").write(data)
    t = time.perf_counter(); r1 = sha256_file("/tmp/big.bin"); t1 = time.perf_counter() - t
    print(f"hashlib (OpenSSL) 50 MB: {t1:.3f}s  {50/t1:.0f} MB/s")
    d2 = data[:2 * 1024 * 1024]
    t = time.perf_counter(); s = SHA256(); s.update(d2); s.digest(); t2 = time.perf_counter() - t
    print(f"pure-python 2 MB: {t2:.2f}s -> {2/t2:.2f} MB/s (50 MB est {50*t2/2:.0f}s)")
```

(c) Optimisation for large files (≥ 50 MB) – measured results

- **Streaming:** the file is read in 1 MiB chunks, so memory stays constant (reading a 50 MB file into one bytes object, or concatenating `data += chunk`, is $O(n^2)$).
- **Pure-Python hot loop:** inlined rotations (no function call per rotation), one `struct.unpack` per block, `memoryview` slices (no copies): **0.28 → 0.44 MB/s (≈1.6×)** on the same data.
- **Production path:** `hashlib.sha256` (OpenSSL, C / SHA-NI): **50 MB in 0.05 s ≈ 1000 MB/s**, about 2000–3000× faster than pure Python (which would need roughly 2 minutes for 50 MB).

```
all test vectors OK  (empty, "abc", 55/56/64-byte boundary cases, 1000 random bytes, byte-by-byte updates)
hashlib (OpenSSL) 50 MB: 0.050s  997 MB/s
pure-python SHA256 class: 0.44 MB/s  (naive version: 0.28 MB/s)
```

(d) Secure hash vs. basic checksum – integrity

Property	Checksum (parity, Adler-32, CRC-32)	Secure hash (SHA-256)
Design goal	Detect *accidental* noise	Resist *deliberate* tampering
Output size	16–32 bits (collision after ~2^{16} files by birthday bound)	256 bits (collision work ≈ 2^{128}, preimage ≈ 2^{256})
Structure	Linear: an attacker changes data and appends 4 bytes to restore the same CRC	Non-linear; avalanche effect (1 flipped bit changes ~50 % of output bits)
Forgery	Trivial	Computationally infeasible; with a key (HMAC-SHA-256) it also gives authenticity
Use	Network frames, disk blocks	Digital signatures, file integrity, software updates, certificates

Therefore a checksum only proves that the file was not damaged by noise, whereas a secure hash proves that no one could have modified it without the digest changing, provided the digest itself is delivered over a trusted channel (or signed). MD5 and SHA-1 are no longer acceptable because practical collisions exist; SHA-256/SHA-3 should be used.

2. Optimization of Secure Socket Layer (SSL/TLS) Simulation

*Program: **q2_tls.py** (TLS 1.3-style handshake: X25519 ECDHE, Ed25519 certificates, HKDF, AES-GCM session tickets).*

(a) Debugging certificate validation and key exchange

#	Defect and why it causes failures / insecurity	Fix
1	Certificate signature never verified, so any self-made certificate is accepted (MITM).	Verify the CA signature over the canonical certificate body.
2	Validity period compared with a strict `not_before <= now` (and local clock skew): **valid certificates randomly rejected**; expired ones sometimes accepted.	Check `nb ≤ now ≤ na`; issue certificates with 60 s back-dating tolerance.
3	Hostname / subject not compared with the requested server.	Reject on `sub != hostname`.
4	Server never proves it owns the certificate's private key.	Server signs the handshake transcript hash (CertificateVerify); client verifies.
5	Static / reused DH key and the raw shared secret used as the key: no forward secrecy; client and server hashed fields in different order or encoding, so **keys sometimes differ** and Finished fails.	Fresh X25519 key per handshake; HKDF over one fixed-order transcript hash; constant-time key confirmation (`hmac.compare_digest`).
6	Weak or low-order public keys accepted.	Reject an all-zero shared secret.
7	Latency: RSA-2048 key generation / signing on every connection, 2 round trips, sleeps and retries after failures.	ECC (X25519/Ed25519), 1-RTT hello carrying the key share, caching and tickets (below).

(b) Optimised code with session resumption

```
"""Q2: TLS-1.3-style handshake simulation: validation fixes + session resumption."""
import os, json, time, hmac, hashlib
from cryptography.hazmat.primitives.asymmetric.ed25519 import Ed25519PrivateKey, Ed25519PublicKey
from cryptography.hazmat.primitives.asymmetric.x25519 import X25519PrivateKey, X25519PublicKey
from cryptography.hazmat.primitives.kdf.hkdf import HKDF
from cryptography.hazmat.primitives.ciphers.aead import AESGCM
from cryptography.hazmat.primitives import hashes, serialization as ser
from cryptography.exceptions import InvalidSignature

RAW = dict(encoding=ser.Encoding.Raw, format=ser.PublicFormat.Raw)
class HandshakeError(Exception): pass

def hkdf(secret, salt, info, n=32):
    return HKDF(hashes.SHA256(), n, salt, info).derive(secret)

# ---------- PKI ----------
class CA:
    def __init__(self): self.key = Ed25519PrivateKey.generate(); self.pub = self.key.public_key()
    def issue(self, host, pub, days=365):
        body = {"sub": host, "pub": pub.public_bytes(**RAW).hex(),
                "nb": time.time() - 60, "na": time.time() + days * 86400}
        raw = json.dumps(body, sort_keys=True).encode()
        return {"body": body, "sig": self.key.sign(raw).hex()}

def validate_cert(cert, ca_pub, hostname, now=None):
    """FIX: signature, validity window AND hostname are all checked (fail closed)."""
    now = now or time.time()
    raw = json.dumps(cert["body"], sort_keys=True).encode()
    try: ca_pub.verify(bytes.fromhex(cert["sig"]), raw)
    except InvalidSignature: raise HandshakeError("bad CA signature")
    b = cert["body"]
    if not (b["nb"] <= now <= b["na"]): raise HandshakeError("certificate expired / not yet valid")
    if b["sub"] != hostname: raise HandshakeError("hostname mismatch")
    return Ed25519PublicKey.from_public_bytes(bytes.fromhex(b["pub"]))

# ---------- Server ----------
class Server:
    def __init__(self, ca, host):
        self.host = host; self.key = Ed25519PrivateKey.generate()
        self.cert = ca.issue(host, self.key.public_key())
        self.ticket_key = AESGCM.generate_key(128)           # only the server can read tickets
    def hello(self, c_rand, c_pub, ticket=None):
        if ticket:                                           # ---- RESUMED (no public-key ops) ----
            try:
                blob = AESGCM(self.ticket_key).decrypt(ticket[:12], ticket[12:], b"ticket")
                psk, exp = blob[:32], int.from_bytes(blob[32:], "big")
                if exp < time.time(): raise ValueError
            except Exception: ticket = None                  # fall back to full handshake
        s_rand = os.urandom(32)
        if ticket:
            keys = hkdf(psk, c_rand + s_rand, b"resumed-keys", 64)
            return {"mode": "resumed", "rand": s_rand}, keys
        eph = X25519PrivateKey.generate(); s_pub = eph.public_key().public_bytes(**RAW)
        shared = eph.exchange(X25519PublicKey.from_public_bytes(c_pub))
        if shared == bytes(32): raise HandshakeError("low-order point")   # FIX: reject weak DH
        transcript = hashlib.sha256(c_rand + c_pub + s_rand + s_pub).digest()
        sig = self.key.sign(transcript)                      # proves possession of cert key
        keys = hkdf(shared, transcript, b"handshake-keys", 64)
```

```
        psk = hkdf(shared, transcript, b"resumption")        # new ticket for next time
        nonce = os.urandom(12)
        t = nonce + AESGCM(self.ticket_key).encrypt(
            nonce, psk + int(time.time() + 3600).to_bytes(8, "big"), b"ticket")
        return {"mode": "full", "rand": s_rand, "pub": s_pub, "cert": self.cert,
                "sig": sig, "ticket": t, "psk": psk}, keys

# ---------- Client ----------
class Client:
    def __init__(self, ca_pub): self.ca_pub = ca_pub; self.cache = {}
    def connect(self, server, host):
        c_rand = os.urandom(32); eph = X25519PrivateKey.generate()
        c_pub = eph.public_key().public_bytes(**RAW)
        cached = self.cache.get(host)
        msg, srv_keys = server.hello(c_rand, c_pub, cached and cached["ticket"])
        if msg["mode"] == "resumed":
            if not cached: raise HandshakeError("unexpected resumption")
            keys = hkdf(cached["psk"], c_rand + msg["rand"], b"resumed-keys", 64)
        else:
            pub = validate_cert(msg["cert"], self.ca_pub, host)
            transcript = hashlib.sha256(c_rand + c_pub + msg["rand"] + msg["pub"]).digest()
            try: pub.verify(msg["sig"], transcript)
            except InvalidSignature: raise HandshakeError("bad handshake signature")
            shared = eph.exchange(X25519PublicKey.from_public_bytes(msg["pub"]))
            keys = hkdf(shared, transcript, b"handshake-keys", 64)
            self.cache[host] = {"ticket": msg["ticket"], "psk": msg["psk"]}
        if not hmac.compare_digest(keys, srv_keys): raise HandshakeError("key mismatch")  # Finished
        return msg["mode"], keys

if __name__ == "__main__":
    ca = CA(); srv = Server(ca, "bank.example"); cli = Client(ca.pub)
    print(cli.connect(srv, "bank.example")[0], cli.connect(srv, "bank.example")[0])
    for bad, exp in ((lambda: validate_cert(srv.cert, ca.pub, "evil.example"), "hostname"),
                     (lambda: validate_cert(srv.cert, ca.pub, "bank.example", now=time.time() + 9e8), "expired"),
                     (lambda: validate_cert(srv.cert, CA().pub, "bank.example"), "bad CA")):
        try: bad(); print("NOT REJECTED!")
        except HandshakeError as e: print("rejected:", e)
    N = 300
    t = time.perf_counter()
    for _ in range(N): Client(ca.pub).connect(srv, "bank.example")
    full = (time.perf_counter() - t) / N * 1000
    c = Client(ca.pub); c.connect(srv, "bank.example")
    t = time.perf_counter()
    for _ in range(N): c.connect(srv, "bank.example")
    res = (time.perf_counter() - t) / N * 1000
    print(f"full {full:.3f} ms   resumed {res:.3f} ms   speed-up {full/res:.1f}x")
```

(c) Results

```
full resumed                                <- first connection full, second resumed from the ticket
rejected: hostname mismatch / certificate expired / bad CA signature   (all three attacks blocked)
full handshake 0.477 ms CPU   resumed 0.050 ms CPU   speed-up 9.6x
```

- **Session resumption:** the server encrypts a resumption secret and expiry into a ticket with its own key (stateless server). The client presents it, and both sides derive new traffic keys with HKDF(psk, client_random + server_random) – **no certificate check, no ECDH, no signature**. New randoms make every resumed session's keys unique, so keys are never reused.
- **Latency:** a full TLS 1.2 handshake costs 2 RTT, TLS 1.3 costs 1 RTT, a resumed session 0–1 RTT. On a 100 ms link this saves 100–200 ms per connection, on top of the 9.6× CPU saving.

(d) Performance vs. security trade-offs

Optimisation	Benefit	Security cost and mitigation
Session tickets / resumption	~10× less CPU, fewer RTTs	If the ticket key leaks, every ticket can be decrypted (weaker forward secrecy). Rotate ticket keys (≤ 24 h), short lifetime (1 h), optionally mix a fresh ECDHE (PSK-DHE).
0-RTT early data	Saves a full RTT	Replayable. Allow only idempotent requests, add anti-replay cache.
Certificate / OCSP caching	Skips validation work	Revocation seen late. Use OCSP stapling with short validity.
Smaller / faster curves, AES-GCM	Lower CPU and latency	Never go below ~128-bit security; ECC is already strong at 256 bits.
Larger session cache	More resumptions	Memory exhaustion / DoS; bound cache size and tickets per client.

Conclusion: resumption and TLS 1.3 give most of the speed-up with a small, controllable security cost, provided ticket keys are rotated, lifetimes are short and certificate validation is never skipped to gain speed.

3. Debugging and Optimizing Digital Signature Algorithm (DSA)

Program: ***q3_dsa.py*** *(1024-bit p, 160-bit q, SHA-256).*

(a) Errors identified

#	Defect (root cause of invalid signatures)	Fix
1	Parameters: q not prime or q □ (p−1); **g chosen as a random number** instead of $h^{(p-1)/q}$ mod p, so g does not have order q and v ≠ r in verification.	Generate prime q, p = kq + 1 prime, g = $h^{(p-1)/q}$ mod p with g > 1 and $g^q \equiv 1$.
2	Private key x and per-message secret k taken from `random` (Mersenne Twister, time seed): predictable. k outside [1, q−1].	`secrets` (CSPRNG); k hedged with HMAC(x, H(m) || fresh random).
3	Same k reused / k derived from a counter.	Fresh k per signature (demo below shows the key is lost otherwise).
4	$s = k^{-1}(H(m)+xr)$ computed with the inverse **mod p** or without final `% q`.	Inverse and reduction both mod q.
5	Hash not truncated to the bit-length of q; signer and verifier use different z.	z = leftmost N bits of the digest (`H()`).
6	r = 0 or s = 0 not retried; verifier does not check 0 < r,s < q.	Loop until r, s ≠ 0; range check in `verify()`.
7	Verification applies `% q` to the wrong quantity: must be `((g^u1 y^u2) mod p) mod q` with $w = s^{-1}$ mod q.	Corrected formula.

(b) Corrected and optimised code

```
"""Q3: DSA - correct parameters, secure k, fixed sign/verify, faster arithmetic."""
import hashlib, hmac, secrets, time

def is_prime(n, rounds=40):                     # Miller-Rabin
    if n < 4: return n in (2, 3)
    if n % 2 == 0: return False
    for p in (3, 5, 7, 11, 13, 17, 19, 23, 29, 31, 37):
        if n % p == 0: return n == p
    d, r = n - 1, 0
    while d % 2 == 0: d //= 2; r += 1
    for _ in range(rounds):
        x = pow(secrets.randbelow(n - 3) + 2, d, n)
        if x in (1, n - 1): continue
        for _ in range(r - 1):
            x = x * x % n
            if x == n - 1: break
        else: return False
    return True

def gen_params(L=1024, N=160):
    """FIX: q is an N-bit prime, p = k*q + 1 is an L-bit prime (so q | p-1),
    g = h^((p-1)/q) mod p must have order q (g != 1)."""
    while True:
        q = secrets.randbits(N) | (1 << (N - 1)) | 1
        if is_prime(q): break
    while True:
        p = secrets.randbits(L) | (1 << (L - 1))
        p -= p % (2 * q) - 1                    # p = 1 (mod 2q)
        if p.bit_length() == L and is_prime(p): break
    h = 2
    while pow(h, (p - 1) // q, p) == 1: h += 1
    g = pow(h, (p - 1) // q, p)
    assert pow(g, q, p) == 1 and g > 1
    return p, q, g

def keygen(p, q, g):
    x = secrets.randbelow(q - 1) + 1            # FIX: CSPRNG, 1 <= x <= q-1 (not random.random)
    return x, pow(g, x, p)

def H(m, q):                                    # leftmost min(N, outlen) bits of the hash
    e = int.from_bytes(hashlib.sha256(m).digest(), "big")
    return e >> max(0, 256 - q.bit_length())

def sign(m, x, p, q, g):
    z = H(m, q)
    while True:
        # FIX: fresh, unpredictable k per signature (hedged: CSPRNG bytes mixed with x and hash(m))
        seed = hmac.new(x.to_bytes(32, "big"), hashlib.sha256(m).digest() + secrets.token_bytes(32), "sha512").digest()
        k = int.from_bytes(seed, "big") % (q - 1) + 1
        r = pow(g, k, p) % q
        s = pow(k, -1, q) * (z + x * r) % q     # FIX: modular inverse mod q, reduce mod q
        if r and s: return r, s                 # FIX: retry if r == 0 or s == 0

def verify(m, sig, y, p, q, g):
    r, s = sig
    if not (0 < r < q and 0 < s < q): return False   # FIX: range check
    w = pow(s, -1, q)                             # FIX: inverse mod q (not mod p)
    u1, u2 = H(m, q) * w % q, r * w % q
    v = pow(g, u1, p) * pow(y, u2, p) % p % q      # OPT: built-in 3-arg pow = square & multiply in C
    return v == r

def verify_fast(m, sig, y, p, q, g):
    """OPT: Shamir's trick - one shared square-and-multiply for g^u1 * y^u2."""
    r, s = sig
    if not (0 < r < q and 0 < s < q): return False
```

```
    w = pow(s, -1, q); u1, u2 = H(m, q) * w % q, r * w % q
    gy = g * y % p; acc = 1
    for i in range(max(u1.bit_length(), u2.bit_length()) - 1, -1, -1):
        acc = acc * acc % p
        b = (u1 >> i & 1, u2 >> i & 1)
        if b == (1, 1): acc = acc * gy % p
        elif b == (1, 0): acc = acc * g % p
        elif b == (0, 1): acc = acc * y % p
    return acc % q == r

def recover_key_from_reused_k(m1, s1, m2, s2, q):
    """Why randomness matters: same k twice => private key recovered."""
    (r, a), (_, b) = s1, s2
    k = (H(m1, q) - H(m2, q)) * pow(a - b, -1, q) % q
    return (a * k - H(m1, q)) * pow(r, -1, q) % q

if __name__ == "__main__":
    t = time.perf_counter(); p, q, g = gen_params(); print(f"params {time.perf_counter()-t:.1f}s")
    x, y = keygen(p, q, g)
    msg = b"transfer 100 to Alice"; sig = sign(msg, x, p, q, g)
    assert verify(msg, sig, y, p, q, g) and verify_fast(msg, sig, y, p, q, g)
    assert not verify(b"transfer 900 to Eve", sig, y, p, q, g)
    assert not verify(msg, (sig[0], (sig[1] + 1) % q), y, p, q, g)
    print("valid signatures verify; tampered ones rejected")
    # randomness failure demo: k reused
    k = secrets.randbelow(q - 1) + 1; r = pow(g, k, p) % q
    m1, m2 = b"msg one", b"msg two"
    s1 = (r, pow(k, -1, q) * (H(m1, q) + x * r) % q); s2 = (r, pow(k, -1, q) * (H(m2, q) + x * r) % q)
    print("private key recovered from reused k:", recover_key_from_reused_k(m1, s1, m2, s2, q) == x)
    N = 100
    t = time.perf_counter(); [verify(msg, sig, y, p, q, g) for _ in range(N)]; a = time.perf_counter() - t
    t = time.perf_counter(); [verify_fast(msg, sig, y, p, q, g) for _ in range(N)]; b = time.perf_counter() - t
    print(f"verify: 2xpow {a/N*1000:.2f} ms ; Shamir {b/N*1000:.2f} ms")
    # naive extended-Euclid inverse vs pow(-1)
    def inv(a, m):
        r0, r1, s0, s1 = m, a, 0, 1
        while r1: qq = r0 // r1; r0, r1 = r1, r0 - qq * r1; s0, s1 = s1, s0 - qq * s1
        return s0 % m
    t = time.perf_counter(); [pow(s_, -1, q) for s_ in [sig[1]] * 20000]; c = time.perf_counter() - t
    t = time.perf_counter(); [inv(s_, q) for s_ in [sig[1]] * 20000]; d = time.perf_counter() - t
    print(f"inverse: pow(-1) {c/20000*1e6:.1f} us ; python EEA {d/20000*1e6:.1f} us")
```

(c) Optimisation of modular arithmetic and results

- Built-in three-argument `pow(g, k, p)` (square-and-multiply in C; never compute `g**k % p`, which builds a huge integer).
- `pow(k, -1, q)` for the modular inverse: **9.6 µs vs 16.1 µs** for a hand-written extended Euclid.
- Exponents are reduced mod q (160 bits) instead of mod p (1024 bits): about 6× fewer multiplications per exponentiation.
- **Shamir's simultaneous exponentiation** in verification computes $g^{u1} \cdot y^{u2}$ with one shared squaring chain: **1.47 ms → 1.14 ms (≈23 % faster)**.
- Parameters (p, q, g) are generated once (1 s) and reused by all users; fixed-base precomputation tables can speed up g^k further.

```
valid signatures verify; tampered message / modified s rejected
private key recovered from reused k: True
verify: 2xpow 1.47 ms ; Shamir 1.14 ms        inverse: pow(-1) 9.6 us ; python EEA 16.1 us
```

(d) Impact of randomness quality on signature security

The signature equation $s = k^{-1}(z + xr) \bmod q$ makes k a one-time secret of the same value as the private key x:

- **Repeated k** (same r in two signatures): $x = (s_1 k - z_1)\, r^{-1} \bmod q$, where $k = (z_1 - z_2)(s_1 - s_2)^{-1}$. The demo above recovers the private key from two signatures. Real incidents: Sony PlayStation 3 (constant k, 2010) and the 2013 Android SecureRandom flaw that emptied Bitcoin wallets.
- **Biased or partly leaked k** (a few bits known): lattice (hidden-number-problem) attacks recover x from a few dozen to a few hundred signatures.
- **Predictable generator** (Mersenne Twister, time seed): attacker reproduces k and computes x directly.
- **Remedy:** use a CSPRNG, or deterministic/hedged nonces (RFC 6979 style), and keep a failure-safe RNG health check. Verification needs no randomness, so only *signing* is at risk. Note that NIST FIPS 186-5 now permits DSA only for verifying legacy signatures; new systems should use ECDSA/EdDSA.

4. Debugging Key Management System

*Program: **q4_keystore.py** (AES-256-GCM key wrapping, scrypt-derived KEK, versioned key lifecycle, RBAC, audit log).*

(a) Flaws identified in storage and lifecycle

#	Flaw	Consequence	Fix
1	Keys saved as plaintext / base64 in JSON or pickle	Anyone who reads the file owns every key	Wrap each key with AES-256-GCM under a key-encryption key (KEK)
2	Hard-coded password; ECB mode or fixed IV; no authentication	Pattern leakage, key reuse, silent tampering	Random 96-bit nonce per wrap; GCM tag; AAD = key_id:version (blob-swap detected)
3	Weak KDF (MD5 / unsalted SHA-256)	Fast offline guessing of the passphrase	scrypt ($N = 2^{15}$, r = 8) with a random 16-byte salt
4	Keys from `random` / timestamp	Predictable keys	`secrets.token_bytes(32)`
5	File mode 0644, non-atomic write	World-readable, corrupt on crash	`chmod 0600`, temp file + `os.replace`
6	No versions, expiry, rotation or revocation	Compromised key lives forever; rotation breaks old data	States ACTIVE → DECRYPT_ONLY → DESTROYED; versions kept for decryption; expiry enforced
7	Every caller can read/write every key; no logging	No least privilege or traceability	Role policy (admin / app / auditor) and audit trail of ALLOW/DENY

(b) Corrected code

```
"""Q4: Key management - encrypted-at-rest keystore, lifecycle, access control, audit."""
import os, json, time, secrets, base64, tempfile
from cryptography.hazmat.primitives.kdf.scrypt import Scrypt
from cryptography.hazmat.primitives.ciphers.aead import AESGCM
b64 = lambda b: base64.b64encode(b).decode()
unb64 = lambda s: base64.b64decode(s)

class AccessDenied(Exception): pass
POLICY = {"admin": {"create", "rotate", "revoke", "read", "list"},
          "app":   {"read", "list"}, "auditor": {"list"}}          # least privilege

class KeyStore:
    def __init__(self, path, passphrase, role="app"):
        self.path, self.role = path, role
        if os.path.exists(path):
            self.db = json.load(open(path)); salt = unb64(self.db["salt"])
        else:
            salt = secrets.token_bytes(16); self.db = {"salt": b64(salt), "keys": {}}
        # KEK derived from passphrase with memory-hard KDF - never stored (FIX: no plaintext/hard-coded key)
        self.kek = Scrypt(salt, 32, 2**15, 8, 1).derive(passphrase.encode())
        self.audit = []

    def _check(self, action, kid=""):
        ok = action in POLICY.get(self.role, ())
        self.audit.append((time.strftime("%F %T"), self.role, action, kid, "ALLOW" if ok else "DENY"))
        if not ok: raise AccessDenied(f"{self.role} may not {action}")

    def _save(self):                                  # atomic + owner-only permissions
        fd, tmp = tempfile.mkstemp(dir=os.path.dirname(os.path.abspath(self.path)))
        with os.fdopen(fd, "w") as f: json.dump(self.db, f)
        os.chmod(tmp, 0o600); os.replace(tmp, self.path)

    def _wrap(self, kid, ver, key):                   # AES-256-GCM, id+version bound as AAD
        n = secrets.token_bytes(12)
        return b64(n + AESGCM(self.kek).encrypt(n, key, f"{kid}:{ver}".encode()))

    def create(self, kid, days=90):                   # generation: CSPRNG, 256-bit
        self._check("create", kid)
        if kid in self.db["keys"]: raise ValueError("exists")
        self.db["keys"][kid] = {"versions": [], "state": "ACTIVE"}
        self._add_version(kid, days); self._save()

    def _add_version(self, kid, days):
        rec = self.db["keys"][kid]; ver = len(rec["versions"]) + 1
        rec["versions"].append({"v": ver, "blob": self._wrap(kid, ver, secrets.token_bytes(32)),
                                "expires": time.time() + days * 86400, "state": "ACTIVE"})

    def rotate(self, kid, days=90):                   # old version -> DECRYPT_ONLY, new version ACTIVE
        self._check("rotate", kid)
        for v in self.db["keys"][kid]["versions"]: v["state"] = "DECRYPT_ONLY"
        self._add_version(kid, days); self._save()

    def revoke(self, kid):                            # crypto-shred: wrapped material deleted
        self._check("revoke", kid)
        for v in self.db["keys"][kid]["versions"]: v["blob"] = None; v["state"] = "DESTROYED"
        self.db["keys"][kid]["state"] = "REVOKED"; self._save()

    def get(self, kid, version=None, for_encrypt=False):
        self._check("read", kid)
        vs = self.db["keys"][kid]["versions"]; v = vs[version - 1] if version else vs[-1]
        if v["state"] == "DESTROYED": raise KeyError("key destroyed")
        if for_encrypt and (v["state"] != "ACTIVE" or v["expires"] < time.time()):
```

```
            raise KeyError("key not active - rotate")
        blob = unb64(v["blob"])                      # InvalidTag => tampering or wrong passphrase
        return v["v"], AESGCM(self.kek).decrypt(blob[:12], blob[12:], f"{kid}:{v['v']}".encode())

if __name__ == "__main__":
    p = "/tmp/ks.json"
    if os.path.exists(p): os.remove(p)
    adm = KeyStore(p, "correct horse battery staple", "admin"); adm.create("db-key")
    raw = open(p).read(); print("file mode:", oct(os.stat(p).st_mode & 0o777))
    v1, k1 = adm.get("db-key"); assert b64(k1) not in raw          # no plaintext on disk
    adm.rotate("db-key"); v2, k2 = adm.get("db-key"); assert (v1, v2) == (1, 2) and k1 != k2
    assert adm.get("db-key", 1)[1] == k1                           # old ciphertexts still decryptable
    try: adm.get("db-key", 1, for_encrypt=True)
    except KeyError as e: print("old version blocked for encryption:", e)
    app = KeyStore(p, "correct horse battery staple", "app")
    try: app.rotate("db-key")
    except AccessDenied as e: print("denied:", e)
    try: KeyStore(p, "wrong passphrase", "app").get("db-key")
    except Exception as e: print("wrong passphrase ->", type(e).__name__)
    d = json.load(open(p)); d["keys"]["db-key"]["versions"][1]["blob"] = d["keys"]["db-key"]["versions"][0]["blob"]
    json.dump(d, open(p, "w"))
    try: KeyStore(p, "correct horse battery staple", "app").get("db-key", 2)
    except Exception as e: print("blob-swap tamper ->", type(e).__name__)
    print(adm.audit[-1])

file mode: 0o600
old version blocked for encryption: 'key not active - rotate'
denied: app may not rotate
wrong passphrase -> InvalidTag            (KEK differs, so unwrap fails)
blob-swap tamper -> InvalidTag            (AAD key_id:version no longer matches)
```

(c) Lifecycle management

- **Generation:** 256-bit CSPRNG key, created only by the *admin* role.
- **Distribution:** never send a raw key; deliver it wrapped (envelope encryption under the recipient's KEK or a DH-derived key) and only to authorised roles.
- **Rotation:** `rotate()` adds version n+1 as ACTIVE and turns older versions DECRYPT_ONLY, so old data stays readable while new data uses the new key (re-encrypt gradually).
- **Revocation / destruction:** wrapped material is deleted (crypto-shredding) and the state becomes REVOKED.

(d) How poor key management compromises an entire cryptosystem

By Kerckhoffs's principle the algorithm is public and all security rests on the keys, so key handling is the weakest link:

- **Single point of failure:** one leaked master or plaintext key decrypts all data (past and future) that it protects, even if AES-256 itself is unbreakable.
- **Weak generation** removes the key space: the 2008 Debian OpenSSL bug left only about 32,000 possible keys; a predictable RNG makes brute force trivial.
- **No rotation or revocation** gives attackers unlimited time with a stolen key and no way to cut them off; a larger blast radius when nothing is compartmentalised.
- **Plaintext, logs, source code and backups** leak keys (e.g. hard-coded keys in repositories, Heartbleed exposing private keys in memory).
- **No access control or audit** means insiders and malware act undetected. Mitigation is defence in depth: envelope encryption with a master key in an HSM/KMS, one key per purpose, least privilege, rotation, auditing and secure destruction.

5. Optimization of Secure File Transfer Protocol (SFTP-like System)

*Program: **q5_sftp.py** (length-prefixed frames, AES-256-GCM per chunk, sequence numbers, final SHA-256 digest frame).*

(a) Errors identified

#	Defect	Effect	Fix
1	Same IV/nonce for every chunk, or counter restarted on each chunk	Keystream reuse breaks confidentiality; sender and receiver desynchronise	Nonce = random 4-byte prefix + 64-bit sequence number (never repeats)
2	Chunk boundaries differ between sender and receiver; `recv(n)` short reads treated as full packets; CBC chaining carried across or reset inconsistently	**Data corruption** that appears only occasionally (when TCP splits a packet)	Length-prefixed frames; `_read_exact()` loop; every chunk independent
3	Encryption without integrity (CBC/CTR only, or a CRC)	Bit flips and malicious modification go unnoticed	AEAD (GCM) tag on each chunk; AAD binds sequence number and session id
4	No ordering or replay checks	Dropped, duplicated or reordered packets accepted	Strict `seq == expected`
5	No end-of-file authentication	Truncation attacks undetected	Final encrypted frame holds SHA-256 of the whole file
6	4 KB reads, byte-wise Python loops, `data += chunk`, separate encrypt and HMAC passes	Very low throughput	Streaming with 64 KiB chunks and AES-NI based GCM (one pass)

(b) Corrected and optimised code

```
"""Q5: SFTP-like transfer - chunked AES-256-GCM, sequence numbers, end-to-end SHA-256."""
import os, struct, hashlib, time, io
from cryptography.hazmat.primitives.ciphers.aead import AESGCM

CHUNK = 1 << 16                                  # 64 KiB: measured sweet spot (4 KiB was slower)
HDR = struct.Struct(">QI")                       # 8-byte sequence no., 4-byte ciphertext length
NONCE = lambda prefix, seq: prefix + struct.pack(">Q", seq)   # FIX: unique nonce per chunk (no reuse)

class IntegrityError(Exception): pass

def send(src, dst, key, session_id):
    """Encrypt-then-frame. GCM tag authenticates data, sequence no. and session (AAD)."""
    aead, prefix, h, seq = AESGCM(key), os.urandom(4), hashlib.sha256(), 0
    dst.write(prefix)
    while block := src.read(CHUNK):
        h.update(block)
        ct = aead.encrypt(NONCE(prefix, seq), block, session_id + struct.pack(">Q", seq))
        dst.write(HDR.pack(seq, len(ct))); dst.write(ct); seq += 1
    ct = aead.encrypt(NONCE(prefix, seq), h.digest(), session_id + b"FIN" + struct.pack(">Q", seq))
    dst.write(HDR.pack(seq, len(ct))); dst.write(ct)         # final frame = whole-file digest

def _read_exact(f, n):                           # FIX: handle short reads (partial packets)
    buf = b""
    while len(buf) < n:
        part = f.read(n - len(buf))
        if not part: raise IntegrityError("truncated stream")
        buf += part
    return buf

def receive(src, dst, key, session_id):
    aead, prefix, h, expect = AESGCM(key), _read_exact(src, 4), hashlib.sha256(), 0
    while True:
        try: seq, n = HDR.unpack(_read_exact(src, HDR.size))
        except IntegrityError: raise IntegrityError("missing final digest frame (truncation)")
        if seq != expect: raise IntegrityError(f"replay/reorder/drop: got {seq}, want {expect}")
        ct = _read_exact(src, n)
        try:
            pt = aead.decrypt(NONCE(prefix, seq), ct, session_id + struct.pack(">Q", seq))
            dst.write(pt); h.update(pt); expect += 1
        except Exception:
            try:                                 # maybe the FIN frame
                digest = aead.decrypt(NONCE(prefix, seq), ct, session_id + b"FIN" + struct.pack(">Q", seq))
            except Exception: raise IntegrityError(f"chunk {seq} failed authentication")
            if digest != h.digest(): raise IntegrityError("file digest mismatch")
            return True

if __name__ == "__main__":
    key, sid = AESGCM.generate_key(256), os.urandom(8)
    data = os.urandom(100 * 1024 * 1024)
    t = time.perf_counter(); wire = io.BytesIO(); send(io.BytesIO(data), wire, key, sid); wire.seek(0)
    out = io.BytesIO(); receive(wire, out, key, sid); dt = time.perf_counter() - t
    assert out.getvalue() == data
    print(f"100 MB encrypt+decrypt+verify: {dt:.2f}s = {100/dt:.0f} MB/s")
    raw = wire.getvalue()
    def attack(mod, name):
        try: receive(io.BytesIO(mod(bytearray(raw))), io.BytesIO(), key, sid); print(name, "NOT DETECTED")
        except IntegrityError as e: print(f"{name}: detected ({e})")
```

```
def flip(b): b[5_000_000] ^= 1; return bytes(b)
attack(flip, "bit flip")
attack(lambda b: bytes(b[:len(b) // 2]), "truncation")
f0 = 4 + HDR.size + CHUNK + 16
attack(lambda b: bytes(b[:4] + b[f0:]), "dropped chunk 0")
```

(c) Results (100 MB file)

```
100 MB encrypt + decrypt + verify: 0.39 s = 253 MB/s   (stream compared byte-for-byte with the original: identical)
bit flip: detected (chunk 76 failed authentication)
truncation: detected (truncated stream)
dropped chunk 0: detected (replay/reorder/drop: got 1, want 0)
chunk-size test (in memory):  4 KiB 225 MB/s | 64 KiB 329 MB/s | 1 MiB 297 MB/s   -> 64 KiB chosen
```

- Throughput of 250–330 MB/s for the *combined* encrypt + decrypt + verify path is above a 1 Gbit/s link (≈125 MB/s), so encryption is not the bottleneck; the network is. Each chunk is processed in a single AEAD pass (no separate HMAC pass), memory stays at one chunk.
- Larger chunks reduce per-call and per-header overhead; beyond about 64 KiB cache effects flatten the gain, so 64 KiB–1 MiB is the sweet spot.

(d) Comparison with insecure transfer (FTP / plain TCP / HTTP)

Property	Insecure transfer (FTP, plain copy)	Optimised secure system
Confidentiality	None: data and passwords visible to a sniffer	AES-256-GCM encryption
Integrity	At best TCP / CRC checksum (accidental errors only)	Per-chunk GCM tag + whole-file SHA-256; any change detected
Authenticity / MITM	None	Key established by authenticated handshake (see Q2)
Replay / reorder / truncation	Not detected	Sequence numbers + final digest frame
Corruption handling	Silent	Fails loudly with IntegrityError; chunk can be re-requested
Speed	~1300 MB/s in-memory copy	~250–330 MB/s; still faster than a gigabit network, so no practical loss

Justification: the optimised design removes the corruption bugs (nonce reuse, short reads, no framing), adds the three missing security services (confidentiality, integrity, authenticity/ordering) and keeps throughput limited by the network rather than by cryptography. The small CPU cost is the price for protection against eavesdropping and tampering that insecure transfer cannot offer.